Section 14.4: Marketplace & Commerce Platforms

> *Part 4 of 8 in the System Design Interview Series*

Executive Overview

Commerce systems demand strong consistency for financial transactions while scaling to handle bursty traffic. Payment systems hold correctness requirements above everything else — inconsistencies lead to real revenue loss and trust destruction. In interviews, the key signals are: understanding when to use eventual vs. strong consistency, knowing the Saga pattern vs. 2PC trade-offs, and articulating idempotency as a first-class design concern.

---

1. Design an E-Commerce Platform (Amazon / Shopify)

Requirements Analysis

Dimension	Requirement
Functional	Product catalog, cart, checkout, order management, inventory
Scale	10M products, 1M concurrent shoppers during peak sale
Transactions	ACID for payments; eventual for catalog reads
Availability	Must survive AZ failures during peak sale events

Consistency Requirements by Component

Component	Consistency Needed	Why	Implementation
Product catalog	Eventual	Stale price for 5s is acceptable	Read replicas + CDN
Inventory count	Strong	Double-selling is catastrophic	Serialisable transactions

Component	Consistency Needed	Why	Implementation
Cart contents	Session-level	Only one user writes to their cart	DynamoDB with conditional writes
Orders	Strong	Must be idempotent, fully durable	ACID relational DB
Payments	ACID	Money — no exceptions	Stripe/bank integration + saga

Cart Lifecycle

```
ACTIVE (browsing) → LOCKED (checkout in progress, 15-min TTL) → ORDERED (paid)
                                                                      ↓
                                                                ABANDONED (30-day
timeout)
```

Locking inventory during checkout: When a cart is locked, decrement `available_count` in inventory. If payment fails, restore the count. Use database-level transactions with timeout to prevent permanent locks.

```
BEGIN;
UPDATE inventory SET available = available - 1
WHERE sku_id = 'X' AND available > 0;  -- Fails if 0 stock

IF affected_rows = 0 THEN ROLLBACK; -- Out of stock
ELSE
  INSERT INTO cart_locks (cart_id, sku_id, locked_until) VALUES (..., NOW() +
INTERVAL '15 min');
COMMIT;
```

Flash Sale Architecture

During a sale, inventory for a popular item can receive millions of concurrent writes. A regular DB row becomes a bottleneck.

Solution — Pre-allocated inventory tokens: 1. Before the sale, write N tokens into a Redis list: `RPUSH sku:123:tokens token_1 token_2 ... token_N` 2. Each purchase atomically pops a token: `LPOP sku:123:tokens` — returns nil if sold out 3. Token acts as a reservation; payment converts it to an order 4. This moves the hot write path off the DB and onto Redis, which handles millions of atomic pops/second

---

Interview Problem 1 — Flash Sale Inventory

"During a flash sale, 100,000 users try to buy the last 500 units of a product simultaneously. How do you prevent overselling while keeping the checkout UX smooth?"

Solution framework:

1. **Redis token pool**: Pre-load 500 tokens into Redis list. Each purchase LPOP a token atomically. No token = sold out. Tokens are fast to check and atomic by design.
2. **Optimistic UI**: Show "Add to Cart" to everyone. Only at checkout do users discover if they got a token. This is better UX than preventing the click.
3. **Token expiry**: If checkout isn't completed in 10 minutes, return the token to the pool via a TTL-based expiry job.
4. **Queue fallback**: For extreme cases (millions competing for 10 items), add users to a virtual queue. Process in order. Users see their position: "You are #4,532 in line."
5. **DB reconciliation**: Periodically reconcile Redis token count against DB inventory count to detect any drift.

---

2. Design a Payment System

Requirements Analysis

Dimension	Requirement
Functional	Authorise, capture, void, refund
Scale	100K transactions/second
Reliability	Exactly-once processing; complete audit trail
Regulatory	PCI-DSS, SOX compliance

Idempotency — The Most Important Pattern

Any payment API endpoint that modifies state must be idempotent. If the network drops after the server processes a payment but before the response reaches the client, the client will retry. Without idempotency, you charge the user twice.

```
Client sends: POST /charges
Header: Idempotency-Key: user_123_order_456_attempt_1

Server:
```

```
1. Check idempotency_keys table for this key
2. If found: return stored response (don't re-process)
3. If new: process payment → store (key, response) → return response

Idempotency key TTL: 24 hours (covers any reasonable retry window)
```

Saga Pattern vs. 2PC

For distributed transactions spanning multiple services (payment + inventory + shipping), you have two options:

Approach	Consistency	Availability	Latency	Complexity	Use When
2-Phase Commit (2PC)	Strong	Low (coordinator SPOF)	High	High	All-or-nothing required, small participant count
Saga (Choreography)	Eventual (compensating txns)	High	Low	Medium	Distributed microservices, long-running flows
Saga (Orchestration)	Eventual	High	Low	High	Complex flows needing central visibility

Payment Saga (Orchestrated):

```
Orchestrator sends:
  Step 1: [Reserve Funds]   → If fail → end
  Step 2: [Authorise Card]  → If fail → [Release Reserve]
  Step 3: [Create Order]    → If fail → [Void Auth] → [Release Reserve]
  Step 4: [Capture Charge]  → If fail → [Cancel Order] → [Void Auth]
  Step 5: [Fulfil]          → If fail → [Refund Capture]
```

Each step's compensating transaction is defined upfront. The orchestrator retries transient failures and executes compensations for permanent failures.

Reconciliation

Daily reconciliation prevents silent discrepancies:

```
For each processor (Stripe, PayPal, etc.):
  1. Fetch their settlement report for yesterday
  2. Compare against internal ledger: Σ(captures) - Σ(refunds) = settlement_amount
  3. Flag discrepancies > $0.01 for manual review
  4. Auto-resolve small floating-point differences within tolerance

Common discrepancy causes:
- Network timeout: charged by processor, no record in internal DB
- Partial refund: internal refund processed, processor not yet settled
- Chargebacks: processor reversal not yet received
```

Interview Problem 2 — Double-Spend Prevention

"Two concurrent payment requests arrive for the same account at the exact same time. Both check the balance, see sufficient funds, and both proceed to debit. The account is overdrafted. How do you prevent this?"

Solution framework:

1. **Database-level serialisation**: Use `SELECT ... FOR UPDATE` or `UPDATE accounts SET balance = balance - amount WHERE id = X AND balance >= amount`. The `balance >= amount` guard fails for the second transaction.

2. **Optimistic locking with version**:

   ```
   UPDATE accounts SET balance = balance - 100, version = version + 1
   WHERE id = X AND version = 42 AND balance >= 100;
   -- If affected_rows = 0: version changed (concurrent update) → retry
   ```

3. **Redis distributed lock**: For very high throughput, acquire a per-account lock before any balance check/debit. Lock held for max 30 seconds. Use Lua script for atomic `lock + check + debit`.

4. **Sharding hot accounts**: Popular merchant accounts receive many concurrent debits. Shard their balance across N sub-accounts. Debit is routed to a random sub-account. Total balance = sum of sub-accounts. Reduces contention N×.

5. **Key insight**: Correctness requires serialisation at some level. Choose where: DB row lock (single node), distributed lock (Redis), or optimistic retry (application level).

---

3. Design a Hotel / Flight Reservation System

Requirements Analysis

Dimension	Requirement
Consistency	Strong — double booking is intolerable
Scale	100K search queries/second; 10K bookings/second
Hot paths	Popular dates/destinations face extreme contention

Booking Pattern Comparison

Pattern	Concurrency	Correctness	User Experience	When to Use
Optimistic locking	High	Risk of retry failure	Retry required	Low-contention resources
Pessimistic locking	Low	Guaranteed	May time out waiting	High-value, short hold period
Two-phase soft reservation	Medium	Guaranteed	15-min provisional hold	User-facing booking flows

Two-Phase Booking (Production Standard)

```
Phase 1 — Soft Reservation (15-minute hold):
  - Mark room as TENTATIVE for user X, expires_at = now + 15min
  - Show user checkout form, price confirmation

Phase 2 — Confirmation (user completes payment):
  - Convert TENTATIVE → CONFIRMED
  - Charge payment method
  - Send confirmation email

Failure paths:
  - User abandons: Soft reservation expires, inventory released automatically
  - Payment fails: Release TENTATIVE reservation, show error
  - Two users compete: First to reach Phase 1 wins; second sees "no availability"
```

Optimistic locking SQL:

```
UPDATE room_inventory
SET available = available - 1, version = version + 1
WHERE room_id = 123
```

```
  AND date = '2025-12-25'
  AND available > 0
  AND version = 7;  -- Fails if concurrent write changed it

-- If affected_rows = 0: retry or return "unavailable"
```

Interview Problem 3 — Preventing Double-Booking Under Load

"Two users try to book the last room at the same hotel for the same date at exactly the same time. Walk through exactly what happens in your system."

Solution framework (walk through at a code level):

```
def book_room(room_id, date, user_id):
    with db.transaction():
        # Row-level lock — serialises concurrent requests for same room+date
        row = db.execute(
            "SELECT available, version FROM room_inventory "
            "WHERE room_id=%s AND date=%s FOR UPDATE",
            (room_id, date)
        ).fetchone()

        if row.available <= 0:
            raise NoAvailabilityError("Room not available")

        # Atomic decrement — second transaction will see available=0
        db.execute(
            "UPDATE room_inventory SET available = available - 1, version =
version + 1 "
            "WHERE room_id=%s AND date=%s",
            (room_id, date)
        )

        # Create reservation record
        db.execute(
            "INSERT INTO reservations (room_id, date, user_id, status) VALUES
(%s, %s, %s, 'CONFIRMED')",
            (room_id, date, user_id)
        )
```

What happens to the two concurrent requests: - Request A acquires the `FOR UPDATE` row lock first - Request B waits (blocked on lock acquisition) - A decrements available from 1 → 0, commits - B acquires lock, reads available = 0, raises NoAvailabilityError - B returns "Sorry, this room is no longer available"

No double-booking possible because the row lock serialises the check-and-decrement operation.

---

4. Design a Ride-Sharing Platform (Uber / Lyft)

Requirements Analysis

Dimension	Requirement
Functional	Real-time dispatch, ETA, pricing, GPS tracking
Scale	10M riders, 3M drivers
Dispatch latency	< 5 seconds to match driver

Geospatial Indexing Comparison

Algorithm	Update Speed	Radius Query	Accuracy	Memory	Use When
Geohash	O(1)	Fast (9-cell lookup)	Good (~±0.5km at 6-char)	Low	Distributed, Redis-friendly
S2 Geometry	O(1)	Fast	Excellent (sphere-based)	Low	Google-scale, complex shapes
R-tree	O(log N)	Medium	Excellent	Higher	PostGIS, complex polygon queries
Brute force	O(1)	O(N)	Perfect	Low	Never at scale

Geohash implementation:

```
# Driver reports location every 5 seconds
GEOADD active_drivers -122.4 37.7 driver:123

# Rider requests a ride from (37.7, -122.4)
GEORADIUS active_drivers -122.4 37.7 5 km WITHDIST ASC COUNT 10
# Returns nearest 10 drivers within 5km, sorted by distance
```

Matching algorithm:

1. Find N nearest available drivers (within 5km)
2. Score each: `score = distance_weight * (1/distance) + rating_weight * rating`

3. Offer to highest-scored driver first; 10-second timeout to accept
4. If declined/timeout: offer to next driver
5. Once accepted: switch driver status to BUSY in Redis

Interview Problem 4 — Surge Pricing Engine

"Design the surge pricing component of a ride-sharing app. When demand exceeds supply, prices should increase dynamically in specific geographic areas."

Solution framework:

1. **Supply/demand signals**: Every 30 seconds, for each geohash cell (5km²): count available drivers and pending ride requests in that cell.
2. **Surge multiplier formula**: `surge = max(1.0, demand / supply)`. Cap at 5× to prevent public backlash. Use logarithmic scale for smoother transitions.
3. **Cell aggregation**: Aggregate nearby cells (3×3 geohash grid) to avoid sharp boundaries where crossing a geohash cell boundary changes price dramatically.
4. **Price update propagation**: Surge multipliers stored in Redis with 60-second TTL. Rider app polls every 30 seconds. Price shown at request time is locked in for 120 seconds (prevents user surprise).
5. **Abuse prevention**: If a driver manipulates surge by logging out to reduce supply, detect this pattern (cluster of nearby drivers going offline simultaneously) and discount their impact on supply calculation.

Section Summary: Commerce Platform Patterns

Scenario	Key Pattern	Watch Out For
Flash sale inventory	Redis token pool	Drift between Redis and DB
Double payment prevention	Idempotency keys + DB FOR UPDATE	Retry storms on failures
Distributed checkout	Saga with compensating transactions	Saga state visibility
Double-booking	Optimistic locking + row lock	Hot rows becoming bottlenecks
Geospatial dispatch	Redis GEORADIUS + scoring	Geohash cell boundary edge cases